



Advance Distributed Systems

Dr. Pummy Dhiman
Assistant Professor
CUIET

- Distributed systems often involve multiple independent components that must collaborate to achieve a common goal. Consider a distributed database replicating data across several servers. If updates to the database are not synchronized, users might see outdated or conflicting information.
- Synchronization in distributed systems refers to the coordination of processes, clocks, or data across multiple nodes to ensure that they operate in a coherent and orderly manner. Without synchronization, systems risk inconsistencies, race conditions, and inefficiencies that can compromise their functionality.
- *Synchronization and coordination are two closely related phenomena.* In process synchronization we make sure that one process waits for another to complete its operation. When dealing with data synchronization, the problem is to ensure that two sets of data are the same.
- When it comes to coordination, the goal is to manage the interactions and dependencies between activities in a distributed system.

Why is synchronization important in distributed systems?

- Synchronization is vital in distributed systems to ensure consistent and coordinated behavior across multiple nodes.
- It helps maintain data integrity, prevent race conditions, and enforce ordering guarantees.
- By synchronizing their actions, distributed nodes can communicate, share resources, and achieve a consistent state despite the challenges of network delays, failures, and concurrency.

Synchronization in distributed systems can be categorized into following primary dimensions:

- Clock Synchronization
- Process Synchronization
- Data Synchronization

- In Distributed Systems, multiple computers (nodes) work together and communicate through a network. Unlike a centralized system, there is no single global clock that all machines perfectly share. Because of this, it becomes difficult to determine:
 - When an event occurred
 - Which event happened first
 - How different processes should coordinate
- Therefore, distributed systems use special clock mechanisms and event ordering techniques to maintain consistency.

Event: An event is any action that occurs in a distributed system.

Examples:

- Sending a message from one process to another
- Receiving a message
- Updating data in a database
- Starting or finishing a computation

Each process in a distributed system generates many events, and these events need to be properly ordered.

Example:

Process P1:

Event A → Send message

Process P2:

Event B → Receive message

Here, Event A must occur before Event B.

- In Distributed Systems, computers are located on different machines and networks.
- Each computer has its own clock, so systems need special ways to manage time.
- There are two main types of clocks:

Physical Clock: A physical clock is the actual clock of a computer that shows real-world time. Example: Laptop clock, Server timestamp

- Example: Server A time $\rightarrow$ 10:00:05, Server B time $\rightarrow$ 09:59:59. Even though both servers are running at the same moment, their clocks may show different times.

Logical Clock: A logical clock does not measure real time. Instead, it only helps to determine the order of events. A logical clock in a distributed system is a mechanism used to order events and capture causality without relying on physical time. It helps in synchronizing events across different nodes to ensure a consistent and coherent sequence of operations.

It answers the question: Which event happened before another event?

Logical clocks use numbers instead of actual time.

Example:

Event A $\rightarrow$ timestamp 1

Event B $\rightarrow$ timestamp 2

Event C $\rightarrow$ timestamp 3

Here we know:

A happened before B

B happened before C

But we don't know the real time.

- In distributed systems, each node runs on its own clock, which can result in time difference. Time in distributed systems is a critical and complex aspect due to the absence of a global clock and the need to coordinate actions across different nodes.
- Time management in distributed systems is crucial for ensuring consistency, coordination, and reliability. While physical clock synchronization provides a basis, logical clocks and consensus algorithms play a significant role in managing the order and causality of events.
- Distributed systems deal with time issues through a combination of synchronization protocols, logical clocks, and consensus algorithms.
- These mechanisms help to ensure that events are ordered consistently, causality is preserved, and the system remains reliable and consistent despite the inherent challenges of distributed environments.

- Clock synchronization in computer networks aims to achieve a consistent notion of time across multiple devices.
- Distributed systems rely on time to order events. Since each machine maintains its own clock, discrepancies can arise due to clock drift (difference in time between clocks of different computers due to small variations in their clock speeds).
- Solution:
 - Physical clock synchronization protocols like Network Time Protocol (NTP) ensure nodes' clocks are aligned.
 - Logical clocks, such as Lamport timestamps and vector clocks, establish event order without relying on physical time.

- **Physical Clocks:** It refers to a real-time clock mechanism that provides the current time based on the system's hardware or an external time source.
 - **NTP (Network Time Protocol):** Used to synchronize the clocks of computers over a network to a few milliseconds of Coordinated Universal Time (UTC).
 - Algorithm:
 - Exchange timestamps between the client and the server.
 - Calculate the round-trip delay and clock offset.
 - Adjust the local clock based on the calculated offset.
 - **PTP (Precision Time Protocol):** Provides higher accuracy than NTP, often used in environments requiring tight synchronization. It relies on hardware timestamping and a master-slave hierarchy.
 - Algorithm:
 - Exchange timestamps between the master and slave devices.
 - Calculate the delay and offset with high precision.
 - Adjust the slave clock based on the calculated offset.

Logical Clocks:

- **Lamport Timestamps:** A method to order events in a distributed system by assigning a numerical timestamp to each event.
- **Vector Clocks:** Extends Lamport timestamps by providing a way to capture causality between events, useful in detecting concurrency and ensuring consistency.

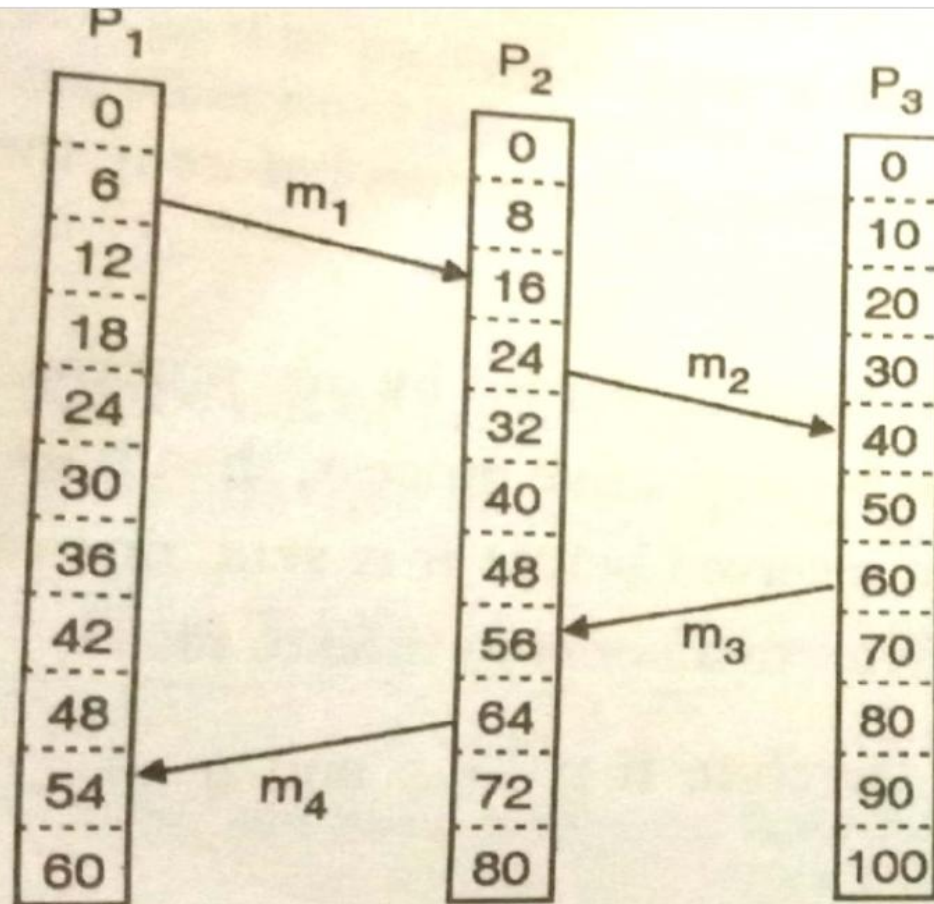
Lamport Timestamps: A method to order events in a distributed system by assigning a numerical timestamp to each event.

- This concept was introduced by Leslie Lamport in 1978 and plays a critical role in ensuring the consistency and coordination of distributed systems.
- To synchronize logical clocks, Lamport defined a relation called **happens-before**. The expression $a \rightarrow b$ is read as ‘a happens before b’ and means that all processes agree that event a occurs then event b occurs. From the definition of the happened-before relation, the clock condition mentioned above is satisfied if the following conditions hold:
 - i. If a and b are two events within the same process and a occurs before b then $a \rightarrow b$ is true
 - ii. If a is message sent by process and b is the receipt of that message by process then $a \rightarrow b$ is also true.
- A message cannot be received before it is sent , or even at the same time it is sent, since it takes finite, non-zero amount of time to arrive.

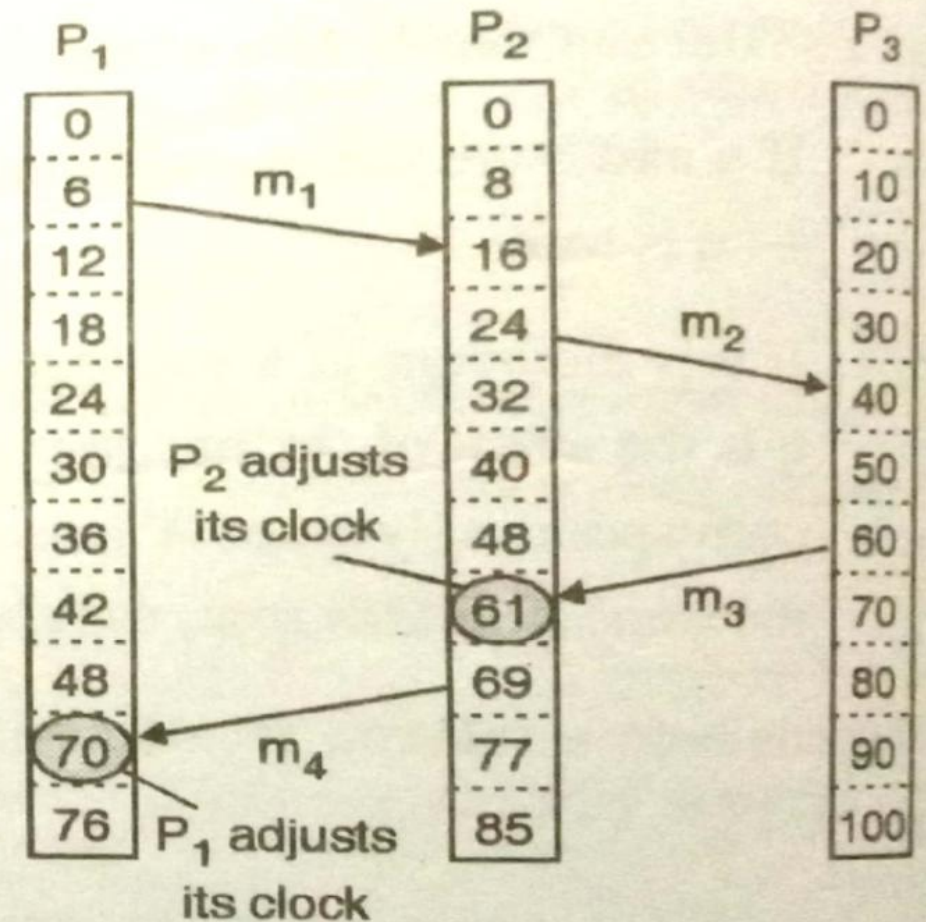
The algorithm follows some simple rules:

- A process increments its counter before each event in that process.
- When a process(a) sends a message to a process(b), it includes its counter value with the message.
- On receiving a message, the counter of the recipient(b) is updated, if clock of receiver is less than clock of sender process: the receiver sets its local clock to $c(b)=c(a)+1$, where $c(a)$ and $c(b)$ are clock of process a and b respectively.
- The counter is then incremented by 1 before the message is considered received.
- Also, each process adjust its clock to their respective interval.
- In other words: When **Process B receives message from A**
- It updates clock using:
 - $C(B) = \max(C'(B), C(A)) + 1$
- Meaning:
 - i. Compare sender clock and receiver clock
 - ii. Take **maximum value**
 - iii. Add **1**

Clock Synchronization: Logical Clocks



(a) Three processes, each with its own clock. The clocks run at different rates.



(b) Lamport's algorithm corrects the clocks.

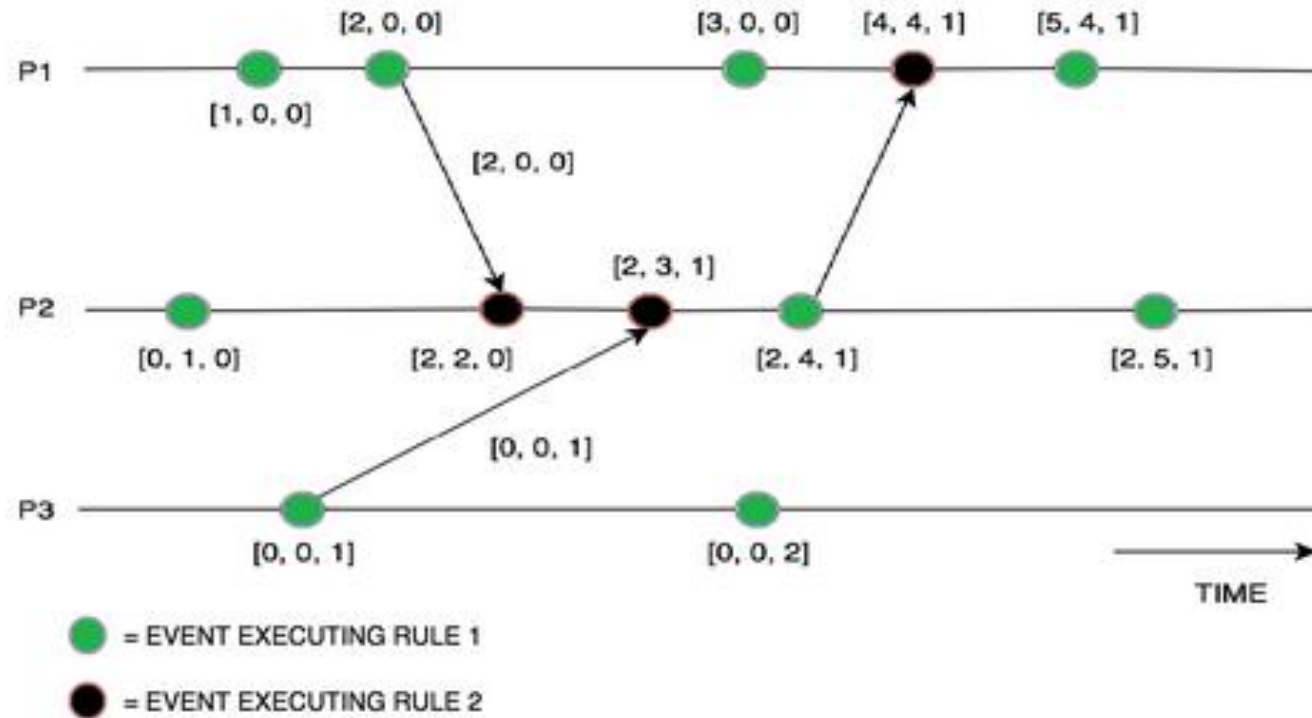
Limitations of Lamport Logical Clock:

- There are some things we don't know since we're not storing information about the state of the other nodes. Lamport clocks cannot tell if two events happened simultaneously (concurrently). It only gives ordering, not true causality.

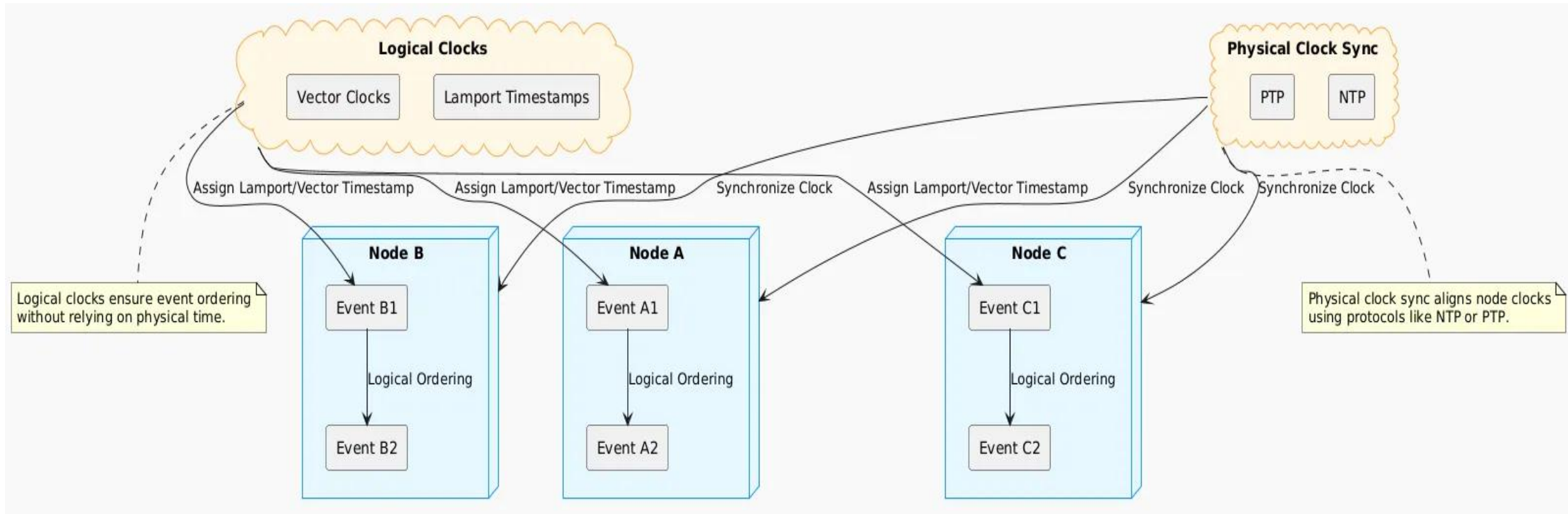
- **Vector clocks** are used in distributed systems to determine whether pairs of events are causally related. With vector clocks, timestamps are generated for each event in the system, and their causal relationships are determined by comparing these timestamps.
- This algorithm labels each process with a vector (or list) containing an integer for each local clock of every process within the system. Therefore, for NNN processes, there will be a vector or array of size NNN .
- In other words: Instead of one timestamp, each process keeps a vector (array) of clocks. Since there are 3 processes, each timestamp has 3 values: $[P1, P2, P3]$
- Example: $[2, 3, 1]$
- Here:
 - P1 knows about 2 events in P1
 - P2 knows about 3 events in P2
 - P3 knows about 1 event in P3

- Initially all clocks are 0.
 - P1 : [0,0,0]
 - P2 : [0,0,0]
 - P3 : [0,0,0]
- *Rule 1*: Internal Event-When a process performs an event
 - increment its own position. P1 event: [0,0,0] $\rightarrow$ [1,0,0]
- *Rule 2*: Send Message-Before sending a message:
 - increment own clock
 - attach vector to message
- *Rule 3*: Receive Message-When message arrives:
 - Take max of each element
 - Increment receiver's own clock

Clock Synchronization: Logical Clocks



Clock Synchronization



Limitations:

- Vector clocks require large timestamp vectors, increase message overhead, do not scale well for large systems, and require knowledge of all processes in the distributed system.

Lamport clocks provide a logical ordering of events, while vector clocks capture the exact causal relationship and detect concurrent events.

- **Process synchronization** coordinates the execution of processes in a distributed system to ensure they operate correctly without conflicts. It ensures that processes or threads coordinate their execution to prevent conflicts, such as two processes accessing the same resource simultaneously.
- This involves managing access to shared resources and preventing issues like race conditions, deadlocks, and starvation.
- A shared resource can be: File, Database etc.
- If two processes access the same resource at the same time, it may cause data inconsistency or system errors. So, synchronization ensures controlled access.

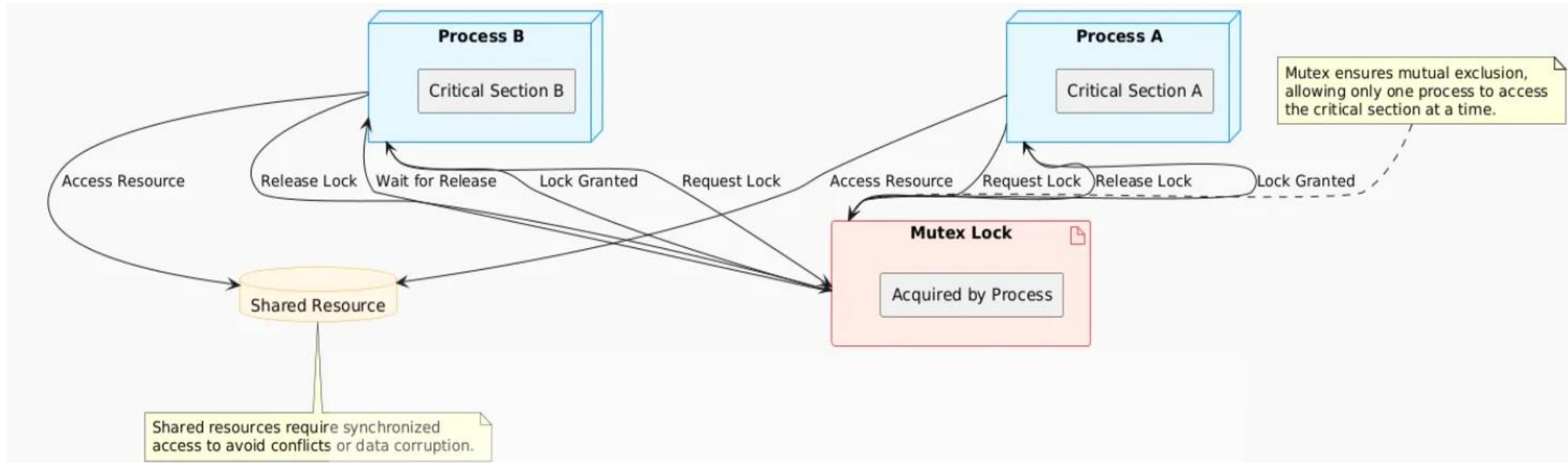
- Importance of Process Synchronization:
 - **Correctness:** Ensures that processes execute in the correct order and interact safely.
 - **Resource Management:** Manages access to shared resources to prevent conflicts and ensure efficient utilization.
 - **Scalability:** Enables the system to scale efficiently by coordinating process execution across multiple nodes.
- Common techniques include:
 - Mutual Exclusion
 - Clock Synchronization
 - Distributed Consensus

Mutex Lock

- Mutex stands for Mutual Exclusion.
- It ensures that only one process can access a critical section at a time.
- When a process acquires the lock, other processes must wait.
- Once the process finishes its work, it releases the lock.

Critical Section:

- A Critical Section is the part of a program where shared resources are accessed.
- Only one process should execute in the critical section at a time.
- If multiple processes enter simultaneously, data corruption may occur.
- Synchronization mechanisms control access to the critical section.



1. Process A and Process B both want to access a shared resource.
2. They send a request to acquire the mutex lock.
3. Only one process (e.g., Process A) gets the lock.
4. Process A enters the critical section and uses the resource.
5. Process B waits until the lock is released.
6. After Process A releases the lock, Process B acquires it and enters the critical section.

Problems Prevented:

- **Race Condition:** when processes access shared data simultaneously.
- **Data Inconsistency:** incorrect or unexpected results.
- **Resource Conflict:** multiple processes competing for the same resource.

Synchronization ensures safe and orderly execution.

- **Data synchronization** ensures that multiple copies of data across different nodes in a distributed system remain consistent. This involves coordinating updates and resolving conflicts to maintain a unified state.

Importance of Data Synchronization:

- **Consistency:** Ensures that all nodes have the same data, preventing inconsistencies.
- **Fault Tolerance:** Maintains data integrity in the presence of node failures and network partitions.
- **Performance:** Optimizes data access and reduces latency by ensuring data is correctly synchronized.

Techniques:

- **Replication:** Copies of data are maintained across multiple nodes to ensure availability and fault tolerance.
- **Consensus Algorithms:** Protocols like Paxos, Raft, and Byzantine Fault Tolerance ensure agreement on the state of data across nodes.

- Maintain consistency and coherence across multiple nodes in distributed systems.
- Enable distributed processes to work together effectively avoiding conflicts and maintaining data integrity.
- Allow efficient resource allocation and load balancing among distributed components.
- Help maintain consistent global state and enable correct ordering of events across different nodes.
- Essential for implementing distributed algorithms (consensus protocols, distributed transactions).
- Address challenges in distributed environments (network partitions, node failures, concurrent access to shared resources).
- Contribute to overall reliability, scalability, and performance of distributed systems

When should one use synchronous communication over asynchronous communication?

- Synchronous communication is suitable when immediate response or strict ordering is required between the sender and receiver. It ensures that the sender waits for a response before proceeding.
- In contrast, asynchronous communication is preferable when non-blocking and parallel processing is desired, allowing the sender to continue with other tasks while waiting for a response. Asynchronous communication can enhance system responsiveness and scalability.

- Mutual exclusion is a property that ensures that a process cannot access a resource while it is already being used by another process.

Or

- Mutual exclusion is a concept in process synchronization used to prevent multiple processes from accessing a shared resource at the same time.

Mutual Exclusion in Computer Systems (Centralized Systems)

- In single computer systems, processes share memory. Mutual exclusion is implemented using shared variables such as:

Semaphores: A semaphore is a synchronization variable used to control access to shared resources. A semaphore is a synchronization tool used in programming to control access to shared resources in multitasking environments.

- A semaphore works by maintaining a counter, which represents the number of available resources. When a process needs a resource, it performs a "wait" operation (decrementing the counter). If the counter is greater than zero, access is granted. If not, the process is blocked until resources become available.
- After using the resource, the process performs a "signal" operation (incrementing the counter), allowing other processes to access the resource.

What is the difference between a semaphore and a mutex?

- A semaphore and a mutex are both used for process synchronization, but they differ in behavior.
- A semaphore can allow multiple threads to access a resource, depending on its counter value, while a mutex only allows one thread at a time. Additionally, semaphores do not require ownership, any thread can signal a semaphore.
- A mutex, however, must be released by the same thread that locked it, ensuring tighter control.

Mutual Exclusion in Distributed Systems

- In distributed systems, processes run on different machines and do not share memory. Therefore, mutual exclusion is implemented using message passing.

Types of Algorithms:

Token-Based Algorithm

In this algorithms, a unique token is shared among all sites. A site is allowed to enter its Critical Section if it possesses the token. The algorithms are:

- Suzuki–Kasami’s Broadcast Algorithm
- Singhal’s Heuristic Algorithm
- Raymond’s Tree-Based Algorithm

Non-Token Algorithm

In this algorithms, timestamps are used to order requests for the Critical Section (CS) and to resolve conflicts between simultaneous requests.

Algorithms:

- Lamport's Algorithm
- Ricart–Agrawala Algorithm
- Maekawa's Algorithm



Thank You